

# UCI F1TENTH Autonomous Racing Stack Technical Specification

Jeffery Lee  
University of California, Irvine  
F1TENTH at UCI

September 2026

## 1 Introduction

---

The RoboRacer (previously F1TENTH) network is a collaborative effort founded at the University of Pennsylvania and led by professor Rahul Mangharam. The goal of RoboRacer is to provide a platform for competitions and research in the field of autonomous racing. The UCI F1TENTH team is a part of this network and has developed our own autonomous racing stack for 1/10th scale racing. In this document, we will provide an overview and guide to our system architecture and software stack, comprising the following: the vehicle, SLAM, and *WarpoRacer*.

In a general scope, the race stack is an end-to-end autonomous navigation platform implemented in ROS 2. The sensor setup currently consists of a 2D Lidar and IMU odometry with future plans to integrate a depth camera. The vehicle carries an NVIDIA Jetson Orin Nano as its computing unit and a VESC Controller for motor controls. Our Simultaneous Localization and Mapping (SLAM) is done by Google Cartographer (under folder `roboracer_slam`). For our autonomous racing approaches, we developed *WarpoRacer*, a highly efficient rigid-body simulator written with NVIDIA Warp for hardware acceleration. *WarpoRacer* allows us to perform rapid iterations of reinforcement learning updates and PPO policy training in the setting of a competition.

## 2 Vehicle Hardware and Sensors

---

Operating 1/10th scale autonomous vehicles at speeds up to 18 m/s requires a reliable chassis. For our purpose, a well-tuned chassis is critical to preserve sensor accuracy. Minimizing the dynamic effects of aggressive turning and acceleration prevents tilting of the LiDAR and other sensors, ensuring consistent readings, as a fault could completely jeopardize the navigation systems.

Table 1: Summary of Core Compute and Perception Hardware

| Category     | Device                                | Primary Function                    |
|--------------|---------------------------------------|-------------------------------------|
| Compute Unit | NVIDIA Jetson Orin Nano (500 GB NVMe) | ROS 2, SLAM, and PPO policy         |
| LiDAR        | Hokuyo UST-10LX Rangefinder           | 2D rangefinder for object avoidance |
| IMU          | 9-Axis IMU                            | Acceleration and rotation           |
| Vision       | Intel RealSense D435i                 | Depth camera for 3D perception      |

## 2.1 Hardware Architecture



Figure 1: The UCI F1TENTH vehicle

- **Compute Module:** The NVIDIA Jetson Orin Nano runs Linux, and is responsible for every computation task that the vehicle performs.
- **Chassis:** The build is based on the 4-wheel drive Traxxas Fiesta ST Rally VXL Edition, with custom parts to allow for mounting of hardware.
- **Tires and Suspension:** The vehicle has an array of tires for varying surface conditions, and a suspension system that allows for spacing adjustments.
- **Motor Control:** We use the VESC 6 MkV TRAMPA controller to control the motor, powered by a 14.8 V (65.1 Wh) LiPo Battery. This also supports our IMU inputs.
- **Sensors:** The primary sensor is a Hokuyo UST-10LX LiDAR scanning a  $270^\circ$  by 10 m rated range (30 m maximum) at 40 Hz. The mount in the front of the vehicle also carries an Intel RealSense D435i depth camera.

### 3 System Architecture and Repository Structure

---

The complete software `race_stack` repository is containerized by Docker for ease of deployment and reproducibility on various OS platforms and cloud servers. Our standard package installer is `astral-uv` for the same purpose. For additional information on how to set up the repository, refer to the `README.md` file.

The software section of the document will follow the chronological order in accordance with running the race stack.

#### 3.1 Simultaneous Localization and Mapping

The SLAM algorithm is currently deployed primarily for generating maps of the race track used for training the PPO policy, since the PPO only requires direct observations to navigate. Localization will be integrated into the pipeline as we explore other methods in future iterations.

- **/roboracer\_slam:** Google Cartographer is used for SLAM, and the ROS 2 node is located in this folder. The ROS node subscribes to the LiDAR and IMU topics, and publishes a map of the environment. The map is generated with loop closures, and is saved to a file for later use.
- **/warpSLAM:** A custom SLAM implementation that utilizes NVIDIA Warp for GPU acceleration. This implementation is still under development.

#### 3.2 Reinforcement Learning

Reinforcement learning methods, such as Proximal Policy Optimization (PPO), require up to hundreds of millions of steps to converge. Standard Gym wrappers fall short in terms of efficiency, as they often render unnecessary graphics and perform physics calculations on the CPU. As a solution, we developed *WarpoRacer*, a GPU-accelerated rigid-body simulator that allows for parallelized simulation of multiple vehicles and race maps.

- **/warporacer:** Our core simulation engine. It is capable of executing simple rigid-body physics and collisions across 8000 parallel instances simultaneously within GPU CUDA kernels. NVIDIA Warp also helps accelerate the computation of 2D LiDAR rays, which are passed into PyTorch networks for PPO policy training.
- **Observation Space:** The observation is a 110 element array that consists of 108 downsampled LiDAR ray casts spanning a  $270^\circ$  field of view along with the vehicle's current steering angle and its forward velocity. Since the PPO policy only sees readings from sensors with no map relevant data, the method requires no localization at run time, resulting in our choice to use SLAM primarily for map generation.
- **Action Space:** There are two values that the PPO policy will output. One being the steering rate and the other being acceleration.
- **Reward Function:** The policy is rewarded for progressing along the track centerline (calculated by the simulator) with a weight dependent on forward velocity. The policy receives

penalties for track wall proximity and deviations off the centerline. Additionally, there is a set penalty for a collision, which will also end the training episode.

- **Visualization:** The simulator is capable of outputting 2D video visualizations of the vehicle's trajectory and other statistics such as episode return. These data can be found in Weights and Biases if the option was selected when running the training script.

### 3.3 Physical Vehicle Deployment

- **/warporacer\_node:** A ROS 2 node that runs the trained PPO policy on the physical vehicle. It subscribes to the `/scan` and the `/odom` topics, and publishes drive commands to `/drive`. The observations are constructed exactly as in training, so that the policy will see matching inputs in simulation and on the physical car.
- **Launching:** Refer to the `README.md` file for specific commands on launching the respective ROS 2 nodes for the desired task.

### 3.4 Safety Constraints and Sanity Checks

Because there are many confounding factors that can cause unexpected behavior on the physical vehicle, we have implemented various safety checks.

- **low\_voltage\_shutdown.py:** A process that shuts the Jetson Orin Nano down if the LiPo battery voltage drops below 12 V.
- **calibrate.py:** Simply sends the vehicle forward for 1 m according to the VESC controller's odometry data. This is measured physically to check the VESC's speed and odometry tuning. Steering is not tuned through this script.
- **Deadman Switch:** A standalone script that does not exist in the repository. It is used to ensure that the vehicle will only run when the operator is actively holding down a button on a controller.

Additionally, we are able to tune some parameters through environment variables without having to rebuild the container. These can be found in the `README.md` file as well.

### 3.5 Simulation Bridge

Since we are using a custom Gym environment, we need a port to the official simulator [AutoDrive](#). The `fitenth_to_autodrive_bridge.py` script translates between the two ROS 2 nodes.

### 3.6 Classical Navigation Stacks

This repository also includes two other classical approaches to navigation, which are not our main research focus, but are still useful for testing purposes.

- **/disparity\_extender:** A LiDAR based reactive approach. It slides a fixed window of LiDAR scans and steers towards the window with the largest disparity (farthest distance to the obstacle).
- **/pure\_pursuit:** A controller that follows some closed race line on the track. This approach requires localization and is therefore implemented but awaiting localization integration.

## 4 Conclusion and Future Outlook

---

This documentation presented the mechanical and software foundations of the UCI F1TENTH autonomous race stack. Through our combination of approaches, we are able to iterate a new version of the PPO policy in under 10 minutes. In the past, this has given us significant operational advantages when the policy fails unexpectedly. In the future, it is necessary to upgrade our simulator from 2D to 3D along with other improvements. This is due to the fact that competitions are evolving to include ramps, which *WarpoRacer* will not be able to sufficiently handle. This will hopefully also improve our physics simulations, as it gives us a chance to be much more precise in our calculations to include parameters such as pitch, roll, vertical load, etc. Lastly, happy racing and welcome aboard!